P_POO



Maksym Tsybulevskyi – CID2A
Lausanne, Vennes
32 périodes
Aurélie Curchaud

Table des matières

1 SPÉCIFICATIONS 3
1.1 TITRE 3
1.2 DESCRIPTION 3
1.3 MATÉRIEL ET LOGICIELS À DISPOSITION 3
1.4 PRÉREQUIS 3
1.5 CAHIER DES CHARGES 3
1.5.1 Objectifs et portée du projet (objectifs SMART) 3
1.5.2 Caractéristiques des utilisateurs et impacts 3
1.5.3 Fonctionnalités requises (du point de vue de l'utilisateur) 3
1.5.4 Contraintes 4
1.5.5 Travail à réaliser par l'apprenti 4
1.5.6 Si le temps le permet … 4
1.5.7 Méthodes de validation des solutions 4
1.6 LES POINTS SUIVANTS SERONT ÉVALUÉS 4
1.7 VALIDATION ET CONDITIONS DE RÉUSSITE 4
2 PLANIFICATION INITIALE 5
3 ANALYSE 5
3.1 OPPORTUNITÉS 5
3.2 DOCUMENT D'ANALYSE ET CONCEPTION 5
3.3 CONCEPTION DES TESTS 7
3.4 PLANIFICATION DÉTAILLÉE 7
4 RÉALISATION 8
4.1 DOSSIER DE RÉALISATION 8
4.2 MODIFICATIONS 9
4.3 UTILISATION DE L'IA DANS CE PROJET 9
5 TESTS 10
5.1 DOSSIER DES TESTS 10
5.2 RAPPORT DE TESTS 10
6 CONCLUSION 11
6.1 BILAN DES FONCTIONNALITÉS DEMANDÉES 11
6.2 BILAN DE LA PLANIFICATION 11
6.3 BILAN PERSONNEL 11
7 DIVERS 12
7.1 JOURNAL DE TRAVAIL 12
7.2 BIBLIOGRAPHIE 12
7.3 WEBOGRAPHIE 12
8 ANNEXES 12

1 SPÉCIFICATIONS

1.1 Titre

Gestion de parking

1.2 Description

Créer le simulateur du parking en console

1.3 Matériel et logiciels à disposition

- Un PC ETML
- Accès à Internet

1.4 Prérequis

- Modules de programmation de base
- ICT-320 en cours

1.5 Cahier des charges

1.5.1 Objectifs et portée du projet (objectifs SMART)

L'objectif de ce projet est de concevoir et développer en C# une application console permettant de simuler la gestion complète d'un parking de 20 places durant les 32 périodes du module ICT-320 à l'ETML.

1.5.2 Caractéristiques des utilisateurs et impacts

L'application s'adresse aux conducteurs souhaitant garer ou récupérer leur véhicule ainsi qu'au gestionnaire du parking. L'interface console utilise un menu numéroté clair et des couleurs (vert, jaune, rouge) pour faciliter la lecture.

1.5.3 Fonctionnalités requises (du point de vue de l'utilisateur)

Le programme propose 6 fonctionnalités principales dans le menu : 1. Entrée d'un véhicule (avec validation de la plaque et création d'un ticket), 2. Sortie d'un véhicule (avec simulation de la durée, calcul du prix et confirmation), 3. Affichage du plan du parking, 4. Recherche d'un véhicule (par plaque ou numéro de place), 5. Statistiques du jour, et 6. Historique des transactions.

1.5.4 Contraintes

Le projet est développé en C# (.NET 9.0) en respectant la programmation orientée objet (classes Parking, ParkingSpot, Vehicule, Ticket). Le code est sauvegardé et versionné régulièrement sur GitHub.

1.5.5 Travail à réaliser par l'apprenti

L'apprenti doit livrer le code source C# fonctionnel et commenté, les tests unitaires MSTest, le journal de travail à jour ainsi que le présent rapport de projet au format PDF via une release GitHub.

1.5.6 Si le temps le permet …

Si le temps le permet, il est possible d'ajouter différents types de véhicules avec des tarifs horaires distincts ou une interface graphique Windows Forms.

1.5.7 Méthodes de validation des solutions

La validation est effectuée par des tests unitaires automatisés avec MSTest (validation des plaques, entrée, sortie et calcul du tarif) ainsi que par des tests fonctionnels manuels dans la console.

1.6 Les points suivants seront évalués

- Le rapport
- Les planifications (initiale et détaillée)
- Le journal de travail
- Le code et les commentaires
- Les documentations de mise en œuvre et d'utilisation

1.7 Validation et conditions de réussite

- Compréhension du travail
- Possibilité de transmettre le travail à une personne extérieure pour le terminer, le corriger ou le compléter
- Etat de fonctionnement du produit livré

2 PLANIFICATION INITIALE

Tâches - objectifs	Nb 1/4 heure	S 1	S 2	S 3	S 4	S 5	S 6	S 7	S 8
Absence - Imprévus	0								
	0								
Docs	27								
	0								
Codding	51								
	0								
Test	9								
	0								
Apprendre	3								
	0								
Fin de Projet	3								
	0								
Absence - Imprévus	0								
	0								
Total planifié	**93**								
Total réalisé	**0**								

3 ANALYSE

3.1 Opportunités

Ce projet représente une opportunité d'approfondir les compétences en programmation orientée objet (POO) et de mettre en pratique de nouvelles méthodes en C#.

3.2 Document d'analyse et conception

Le code va contenir les 4 classes :

Classe Parking

La classe Parking représente le parking dans son ensemble. Elle contient les places de parking et permet de gérer les véhicules présents dans le parking.

Elle sera notamment utilisée pour :

- ajouter un véhicule lorsqu'il entre dans le parking ;
- trouver une place libre ;
- rechercher un véhicule par sa plaque d'immatriculation ou son numéro de place ;
- libérer une place lorsqu'un véhicule sort ;
- afficher l'état actuel du parking ;
- calculer les statistiques du parking.

Le Parking sera donc la classe principale qui coordonne les différentes opérations.

Classe ParkingSpot (Place de parking)

La classe ParkingSpot représente une place individuelle du parking (numéro de place, état IsOccuped, véhicule garé VehiculePark et ticket actif ActiveTicket).

Chaque place possède un numéro permettant de l'identifier et un état indiquant si elle est libre ou occupée. Une place pourra être associée à un véhicule lorsqu'elle est occupée.

Elle sera utilisée pour :

- identifier une place ;
- vérifier si elle est libre ;
- associer un véhicule à une place ;
- libérer la place lorsqu'un véhicule sort.

Classe Vehicule

La classe Vehicule représente un véhicule qui entre dans le parking.

Elle contient notamment la plaque d'immatriculation du véhicule et les informations nécessaires à son stationnement.

Lorsqu'un véhicule entre, sa plaque est saisie et vérifiée. Le parking lui attribue ensuite une place libre et un ticket.

La classe Vehicule permettra également de retrouver un véhicule à partir de sa plaque d'immatriculation.

Classe Ticket

La classe Ticket représente le ticket associé au stationnement d'un véhicule.
Le ticket contient les informations nécessaires pour suivre le stationnement :

- Le lien sur la classe Véhicule ;
- l'heure d'entrée ;
- la place attribuée ;
- le tarif horaire.

Lors de la sortie du véhicule, le ticket sera utilisé pour déterminer la durée du stationnement et calculer le montant à payer. Après confirmation de la sortie, la place associée au ticket sera libérée.

Entrée d'un véhicule

Créer la saisie et la vérification de la plaque d'immatriculation selon le format demandé. Attribuer une place disponible et créer un ticket contenant l'heure d'entrée, la place et le tarif horaire.

Sortie d'un véhicule

Permettre la recherche et la sortie d'un véhicule par plaque ou par numéro de place en utilisant les méthodes FoundVehicle(plaque, place, exitTime) et SortVehicle(plaque, place, exitTime) ;
Afficher l'heure d'entrée, l'heure de sortie, la durée et le montant à payer. Demander une confirmation avant de libérer la place.

État du parking

Afficher les places du parking et indiquer quelles places sont libres (L) et lesquelles sont occupées (X) en utilisant la méthode DisplayParking() ;

Recherche d'un véhicule

Permettre de rechercher un véhicule à l'aide de sa plaque d'immatriculation ou de son numéro de place en utilisant la méthode FoundVehicle(plaque, place). Afficher les informations du ticket lorsque le véhicule est trouvé.

Statistiques du jour

Calculer et afficher le nombre de places occupées, le pourcentage d'occupation, le montant total payé et la durée actuelle de stationnement des véhicules en utilisant la méthode DisplayStatistics().

Historique des transactions

Enregistrer dans la liste Transactions et afficher les entrées et sorties des véhicules avec les informations correspondantes en utilisant la méthode DisplayTransactions().

Fonctionnalités supplémentaires

Si le temps le permet, ajouter une fonctionnalité supplémentaire, par exemple plusieurs types de véhicules avec des tarifs différents.

Contraintes techniques

Mettre en place les classes Parking, ParkingSpot, Vehicule et Ticket. Respecter un nommage cohérent et appliquer les principes de programmation orientée objet.

3.3 Conception des tests

Les tests vont être fait à l'aide d'un outil MSTest, ça permet de faire des tests unitaires.

3.4 Planification détaillée

Semaine	1	
Tâche	**Durée [1/4 h.]**	**Explications:** qu'est-ce qui se fait et comment ?
Docs	6	Creation et remplissage de Planning et Rapport
Codding	3	Debut de codage (Faire l'affichage de debut)
Semaine	**2**	
Tâche	**Durée [1/4 h.]**	**Explications:** qu'est-ce qui se fait et comment ?
Codding	6	Faire des fonctionnalités (entrée du vahicule - sortie d'un véhicule)
Apprendre	3	Se former de fonctionnalités que je ne connais pas
Docs	3	Remplir le JdT et Le Rapport
Semaine	**3**	
Tâche	**Durée [1/4 h.]**	**Explications:** qu'est-ce qui se fait et comment ?
Test	3	Faire de test du jeu
Codding	6	Corriger les erreurs et faire (Rechercher un véhicule)
Docs	3	Remplir le JdT et Le Rapport
Semaine	**4**	
Tâche	**Durée [1/4 h.]**	**Explications:** qu'est-ce qui se fait et comment ?
Codding	9	Faire le point "Statistique du jour" du cahier de charge
Docs	3	Resumer le travail réalisé

Semaine	5	
Tâche	**Durée [1/4 h.]**	**Explications:** qu'est-ce qui se fait et comment ?
Codding	9	Faire le point "Historique des transactions" du cahier de charges
Docs	3	Resumer le travail réalisé
Semaine	**6**	
Tâche	**Durée [1/4 h.]**	**Explications:** qu'est-ce qui se fait et comment ?
Test	3	Tester les dernieres points (qui n'ont pas été testé), avec le code automatique
Codding	6	Corriger des bugs
Docs	3	Resumer le travail réalisé
Semaine	**7**	
Tâche	**Durée [1/4 h.]**	**Explications:** qu'est-ce qui se fait et comment ?
Codding	9	Finir les points "Pour aller au-delà du minimum" et "Contraintes techniques"
Docs	3	Resumer le travail réalisé
Semaine	**8**	
Tâche	**Durée [1/4 h.]**	**Explications:** qu'est-ce qui se fait et comment ?
Test	3	Faire le test global
Codding	3	Corriger des erreurs possibles
Docs	3	Verification des données dans le Rapport et JdT
Fin de Projet	3	Verification de tout ensemble et rassurer que le projet est fini

4 RÉALISATION

4.1 Dossier de Réalisation

1. Environnement et outils utilisés :
- Langage et Framework : C# avec .NET 9.0 (Application Console)
- Environnement de développement (IDE) : Visual Studio / JetBrains Rider (Windows et macOS)
- Framework de tests unitaires : MSTest (Microsoft.NET.Test.Sdk 17.12.0, MSTest 3.6.4)
- Gestion de versions : Git et GitHub

2. Architecture et explication du diagramme de classes (Conception OO) :
Conformément au cahier des charges, l'architecture orientée objet repose sur 4 classes métier principales et la classe Program :

- Vehicule (Vehicule.cs) : Modélise une voiture avec sa propriété LicencePlate (string). Elle contient la méthode statique IsValidPlate(string plate) qui utilise une expression régulière (^[A-Za-z]{2}-\d{1,6}$) pour valider le format de plaque (ex: VD-123456).

- Ticket (Ticket.cs) : Représente le ticket attribué à l'entrée. Il stocke la référence vers le véhicule (VehiculePark), le numéro de place (GivenPlace), l'heure d'entrée (EntryTime) et le tarif horaire (tarifPerHour). Ses méthodes ParkDuration(DateTime exitTime) et SumPayement(DateTime exitTime) calculent la durée de stationnement et le prix à payer (toute heure entamée est due, avec un minimum d'une heure).

- ParkingSpot (ParkingSpot.cs) : Représente une place de parking identifiée par son numéro (Number) et son état (IsOccuped). La méthode Park(Vehicule v, Ticket t) associe un véhicule et son ticket à la place, tandis que Release() libère la place lors de la sortie.

- Parking (Parking.cs) : Classe centrale qui contient un tableau de 20 places (ParkingSpot[] Spots), la capacité (Capacity), le tarif horaire (HourlyRate = 1.5 CHF/h), le montant total encaissé (TotalPaid) et l'historique des mouvements (List<string> Transactions). Elle coordonne toutes les opérations : FindFirstFreeSpot(),

EnterVehicle(), SortVehicle(), DisplayParking(), FoundVehicle(), DisplayStatistics() et DisplayTransactions().

- Program (Program.cs) : Gère l'affichage du menu interactif en console (options 0 à 6), la saisie utilisateur et la simulation des heures de stationnement lors de la sortie.

4.2 Modifications

Au cours de la réalisation, plusieurs améliorations et ajustements ont été apportés par rapport à l'analyse initiale :

- Validation des plaques (18.09 - 24.10.2026) : Adaptation de l'expression régulière dans Vehicule.IsValidPlate() pour imposer le tiret dans le format de plaque (ex: VD-123456) conformément à la maquette du cahier des charges, et ajout d'une vérification dans EnterVehicle() pour empêcher un même véhicule d'entrer deux fois simultanément.

- Simulation de la durée et confirmation de sortie (24.10.2026) : Ajout de la possibilité de simuler un nombre d'heures de stationnement lors de la sortie du véhicule et ajout d'une demande de confirmation (o/n) avant de libérer effectivement la place.

- Historique et statistiques (24.10.2026) : Utilisation d'une collection dynamique List<string> Transactions pour enregistrer chaque entrée et chaque sortie avec son montant, et ajout du cumul TotalPaid pour les statistiques du jour.

4.3 Utilisation de l'IA dans ce projet

Conformément aux directives du cahier des charges (Niveau 2 – Planification avec l'IA), l'intelligence artificielle a été utilisée de manière encadrée et pédagogique comme un assistant de vérification, de planification et d'explication de concepts tout au long de la finalisation du projet.

Premièrement, au début de la réalisation des classes métier (18 septembre 2026), l'IA a été sollicitée pour générer un script de test global de démonstration (TestGlobalParking_GenereParIA.cs) afin de vérifier rapidement que les quatre classes développées par l'apprenti (Vehicule, ParkingSpot, Ticket et Parking) interagissaient correctement ensemble. Ce fichier a été clairement identifié et documenté dans un dossier séparé Documentation_Tests.

Deuxièmement, lors de la dernière phase du projet, l'IA a été utilisée pour passer en revue le code existant par rapport aux exigences précises du cahier des charges étape par étape. Cette démarche a permis à l'apprenti d'identifier les éléments restant à compléter, tels que la confirmation de sortie du véhicule, la simulation de la durée de stationnement, les statistiques du jour et l'historique des transactions.

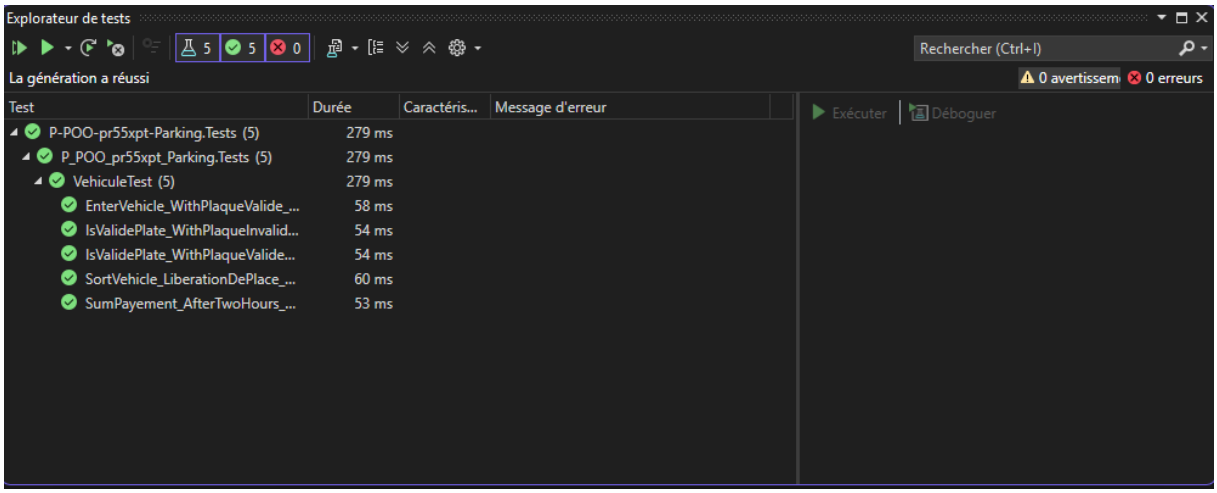
Troisièmement, l'IA a permis d'approfondir la compréhension de certains concepts techniques en C#, notamment le formatage de l'objet TimeSpan (utilisation de TotalHours, Minutes et Seconds pour éviter d'afficher les millisecondes), l'arrondi du tarif horaire avec Math.Ceiling et Math.Round, l'adaptation de l'expression régulière Regex pour le format VD-123456. Chaque modification a été comprise, testée et intégrée étape par étape par l'apprenti dans le code.

5 TESTS

5.1 Dossier des tests

Date	Descritpion	Validation
02.10.2026	Vérifier que la méthode IsValidPlate() retourne true avec une plaque valide au format VD-123456 : IsValidePlate_WithPlaqueValide_Result_True() Entrée : "VD-123456" -> Résultat attendu : true	Ok
02.10.2026	Vérifier que la méthode IsValidPlate() retourne false avec une plaque invalide : IsValidePlate_WithPlaqueInvalide_Result_False() Entrée : "q" -> Résultat attendu : false	Ok
02.10.2026	Vérifier que la méthode SortVehicle() libère bien la place et retourne le ticket correspondant : SortVehicle_LiberationDePlace_ResultTrue() Entrée puis sortie de "VD-123456" -> Résultat attendu : Ticket non null	Ok
24.10.2026	Vérifier que la méthode EnterVehicle() attribue une place libre et crée un ticket valide : EnterVehicle_WithPlaqueValide_ResultNotNull() Entrée : "VD-123456" -> Résultat attendu : Ticket non null (Place n°1)	Ok
24.10.2026	Vérifier le calcul du prix du stationnement avec SumPayement() pour 2 heures à 1.50 CHF/h : SumPayement_AfterTwoHours_ResultThreeChf() Durée : 2h, Tarif : 1.5m -> Résultat attendu : 3.0 CHF	Ok

5.2 Rapport de tests



Les 5 tests unitaires implémentés avec MSTest dans la classe VehiculeTest passent tous avec succès (5/5 réussis). En complément, des tests fonctionnels manuels ont été réalisés sur les 6 options du menu principal (entrée, refus de doublon, sortie avec simulation d'heures et confirmation, affichage de la grille des 20 places, recherche par plaque/place, statistiques du jour et historique des transactions).

6 CONCLUSION

6.1 Bilan des fonctionnalités demandées

Toutes les fonctionnalités demandées dans le cahier des charges ont été réalisées à 100 % :

- 7.3.1 Entrée d'un véhicule : Saisie et validation de la plaque par Regex (ex: VD-123456), vérification des doublons, attribution de la première place libre et affichage du ticket (100 % atteint).

- 7.3.2 Sortie d'un véhicule : Recherche par plaque ou numéro de place, simulation de la durée de stationnement en heures, affichage du récapitulatif complet (entrée, sortie, durée, prix) et demande de confirmation (o/n) avant libération de la place (100 % atteint).

- 7.3.3 Afficher l'état du parking : Affichage visuel du plan des 20 places indiquant les places libres (L) et occupées (X) (100 % atteint).

- 7.3.4 Rechercher un véhicule : Recherche par plaque ou par numéro de place et affichage des détails du ticket et du prix actuel (100 % atteint).

- 7.3.5 Statistiques du jour : Affichage du nombre de places occupées, du taux d'occupation en %, du montant total payé en CHF et de la durée totale actuelle de stationnement (100 % atteint).

- 7.3.6 Historique des transactions : Enregistrement et affichage chronologique de toutes les entrées (ENTREE) et sorties (SORTIE) avec les montants payés (100 % atteint).

6.2 Bilan de la planification

Le projet s'est déroulé sur les 32 périodes prévues. L'analyse fonctionnelle, la conception des 4 classes (Parking, ParkingSpot, Vehicule, Ticket) et les fonctionnalités de base ont été mises en place conformément à la planification initiale.
Durant la deuxième partie du projet, un léger décalage a eu lieu lors de l'implémentation de la sortie d'un véhicule (simulation du temps et calcul du tarif) ainsi que pour les statistiques et l'historique des transactions. Ces points ont pu être finalisés et testés dans les délais avant la livraison finale du 30 octobre 2026, comme détaillé dans le journal de travail.

6.3 Bilan personnel

Ce projet m'a permis de consolider ses connaissances en programmation orientée objet (POO) en C#. J'ai notamment permis de structurer une application autour de plusieurs classes qui collaborent entre elles (Parking, ParkingSpot, Vehicule et Ticket), d'utiliser les expressions régulières (Regex), de manipuler les dates et durées (DateTime et TimeSpan), de gérer une liste dynamique (List<string>) et de rédiger des tests unitaires avec MSTest.

Si ce projet devait être refait, davantage de temps serait consacré dès le départ à la définition exacte du format d'affichage des durées ainsi qu'à la rédaction des tests unitaires au fur et à mesure du développement de chaque méthode. Comme amélioration future, il serait pertinent d'ajouter différents types de véhicules (motos,

voitures, camions) avec des tarifs adaptés ou une interface graphique Windows Forms.

7 DIVERS

7.1 Journal de travail

Le journal de travail complet et détaillé (incluant toutes les tâches, durées, commits et prompts IA) est fourni dans le fichier annexe T-POO-pr55xpt.xlsm.

7.2 Bibliographie

Support de cours ETML – Module ICT-320 : Programmation orientée objet.

7.3 Webographie

- Documentation officielle Microsoft C# / .NET : https://learn.microsoft.com/fr-fr/dotnet/csharp/
- Documentation TimeSpan et DateTime en C# : https://learn.microsoft.com/fr-fr/dotnet/api/system.timespan
- Testeur d'expressions régulières : https://regex101.com/

8 ANNEXES

- Code source complet du projet et des tests unitaires dans le dépôt GitHub (solution P-POO-pr55xpt-Parking.sln).
- Journal de travail : T-POO-pr55xpt.xlsm.